# Prompt: Streamlit IMU/EKF State Dashboard

## Context

I already have working Python classes for reading and processing IMU data over UART:

- **UARTFPGA_reader.py** — reads the raw binary UART stream from the device via read_packet
- **Preparer** — parses/prepares raw frames into usable IMU samples
- **Converter** — converts prepared samples into the format the filter expects
- **EKF** — the Extended Kalman Filter, producing the state vector, covariance matrix, innovation, and a mag validity flag

These four classes are attached below/in the codebase. Treat them as a fixed backend library — do not rewrite their internals. The filter design itself is finished; only minor tuning remains on the mag and psi (yaw) angle handling.

- there is also a functioning app that just plainly prints the data to the terminal. u can take it as a guide for the backend to know how to use the classes
  This is the **final, production version** of the dashboard, not a prototype.

## Goal

Build a Streamlit dashboard that visualizes the live state of the EKF, with two selectable operating modes.

### Mode 0 — Live IMU + local EKF (current focus)

In this mode there is no onboard EKF or calibration — raw IMU data streams in over UART and the dashboard itself drives the pipeline:

```
UartReader→ Converter  → Preparer → EKF
```

The dashboard must:

1. **Stream at ≥60 Hz refresh rate** for the live display.
2. Show the **latest raw sensor readings**: gyro (3-axis), accel (3-axis), mag (3-axis). if IMU is fresh lamp
3. Show the **current EKF state vector** (all states, clearly labeled).
4. Display the **covariance matrix** in a way that's actually interpretable — not just a raw matrix dump. For example: a heatmap with a diverging colormap, correlation-normalized off-diagonal terms, and/or the diagonal (variances) called out separately from the off-diagonal (correlations). or even arranged in a matrix form that keep updating over time
5. Show **uncertainty evolution over a rolling ~4 second time window**, and also show the **initial uncertainty** (e.g., as a reference line/marker) so tuning progress is visible against the starting point.

6. Provide **at least two uncertainty time-series graphs**: according tot he agreed Display widow.
    - One graph with all **3 Euler angle σ (sigma)** values (roll/pitch/yaw), each state in its own color.
    - One graph with all **3 gyro bias σ (sigma)** values, each state in its own color, extracted from the current covariance matrix.
7. Provide a separate **innovation graph**.
8. Surface the **mag validity flag** from the EKF somewhere visible (e.g., status indicator).
9. Most inportant part a 3 D Quadecopter drone model that updates according to the current euler states

### Mode 1 — Parse-only (FPGA handles the EKF/transition)

Same dashboard, same layout and visualizations, but the pipeline is just:

```
UartReader → Converter → (display)
```

No local EKF is run — the FPGA has already run the filter and handles mode transition logic. **This mode's data path is not implemented yet** — stub it out with a clearly marked placeholder (e.g., a disabled/"Coming soon" panel or a TODO stub function) rather than building it now. and guide me where to adjsut the related part once the pipe line is finished

## Architecture requirements

- **Strict separation between backend/engine and frontend.** The Streamlit UI code should not contain pipeline logic — it should only call into an engine module/class that wraps UartReader → Converter → Preparer →EKF (or the Mode 1 equivalent) and exposes a clean interface (e.g., "get latest sample", "get current state", "get history buffer").
- Use a **bounded rolling-window buffer** for the live history (not an ever-growing list), sized to comfortably cover the 4-second uncertainty window at the target refresh rate. (we can argue about the right window size so this should be a MACRO on the top of the Program to eaily change it)
- File/module structure should be **well organized and easy to maintain** — e.g., separate files for: engine/pipeline wrapper, EKF-facing data models, plotting/visualization helpers, and the Streamlit app/layout itself. Favor small, single-responsibility modules over one large script.
- Mode switching should be a simple, explicit toggle in the UI that swaps which engine is active, without duplicating the display/plotting code between modes.
- all the Constants macros in the app.py are setting also keep them on the top and don't change them jsut re use them
- as u see from the read_packet() funciton in the UARTFPGA_reader.py. it returns also the mode. so this is what dtermineds the mode for u.

## Deliverable

Propose a file structure first, then implement it, keeping the Streamlit front end as a thin presentation layer over the engine module.